

Continuous-Time Approximate Dynamic Programming as an Active Learning Problem

Master Thesis presentation

Balduin Dettling

Supervision: Lenart Treven, Bhavya Sukhija, Andreas Krause, Florian Dörfler

Optimal Control

For a dynamical system $\dot{x} = f(x, u)$, find:

$$\begin{aligned} \min_{\substack{u: [0, \infty) \rightarrow \mathcal{U} \\ x: [0, \infty) \rightarrow \mathcal{X}}} \quad & \int_0^\infty l(x(t), u(t)) dt \\ \text{s.t.:} \quad & \dot{x} = f(x(t), u(t)) \quad \forall t > 0 \\ & x(0) = x_0 \end{aligned}$$

Optimal Control Theory Crash Course

- Problem is solved by *value function* $V(x)$
- Optimal input recovered from value gradient $V_x(x)$
- V is the *viscosity solution* the HJB equation
- Locally optimal trajectories: PMP

Numerical methods for Optimal Control

- Trajectory optimisation
 - Only locally optimal solution
 - Accurate approximation of continuous-time solution
- “Dynamic Programming”
 - As in: Find V that satisfies HJB equation
 - Global optimality
 - Practical methods immature

```
graph TD; A[Goal: Practical numerical method that does both] --> B[Accurate approximation of continuous-time solution]; A --> C[Global optimality];
```

Goal: Practical numerical method that does both

Global Optimality in continuous time?

Decompose problem:

1. Generate large set of *locally* optimal trajectories
2. Extract approximate knowledge of *globally* optimal control law

Trajectory Generation

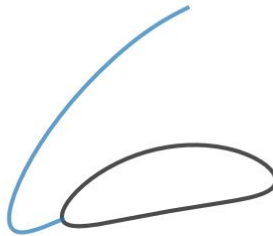
Pontryagin's Minimum Principle

Trajectory Generation



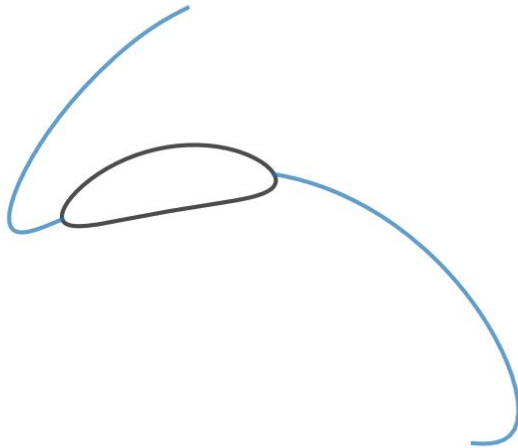
— Value level sets
— Optimal trajectories

Trajectory Generation



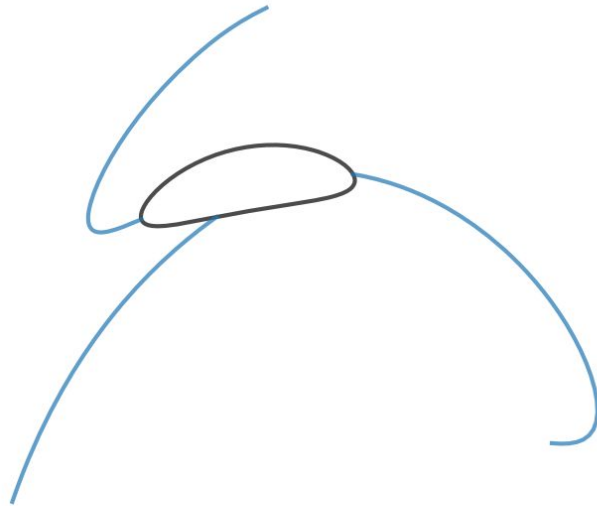
— Value level sets
— Optimal trajectories

Trajectory Generation



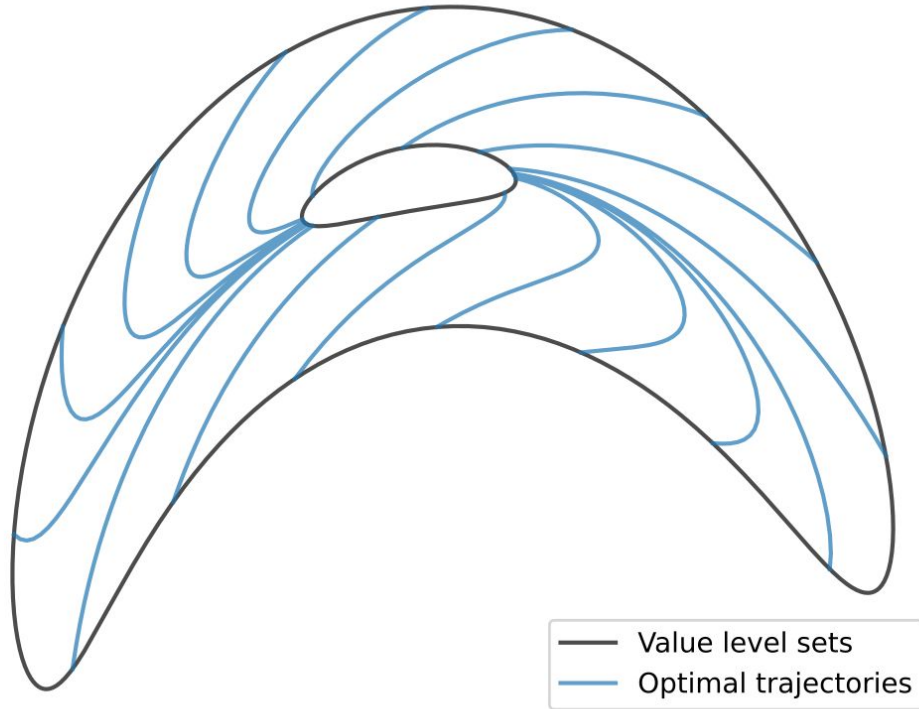
— Value level sets
— Optimal trajectories

Trajectory Generation

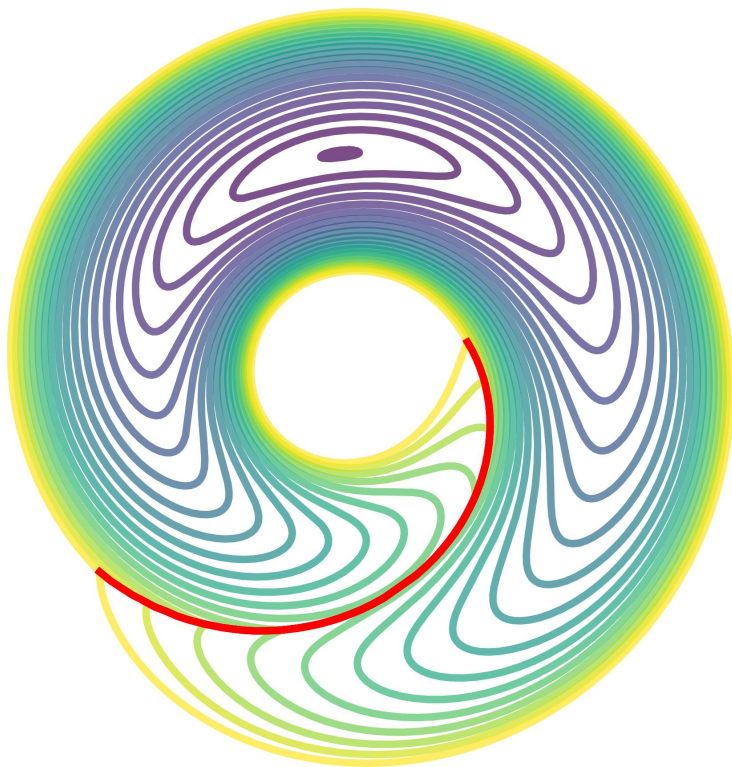


— Value level sets
— Optimal trajectories

Trajectory Generation



Why value level sets?



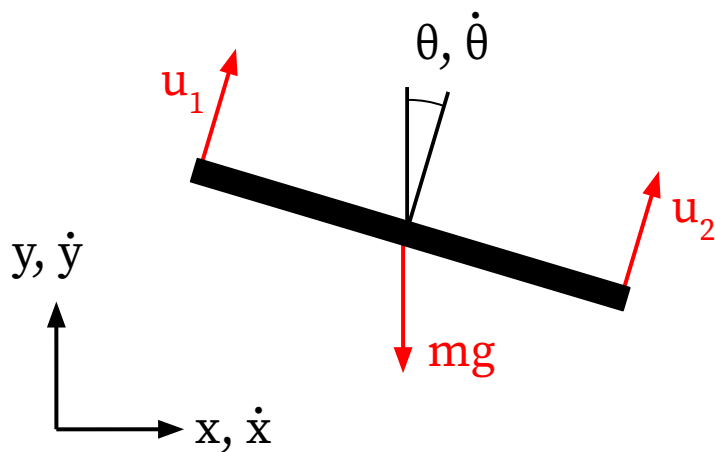
In Practice:

- Finite data
- Uncertain function approximation

Ideas from Active Learning

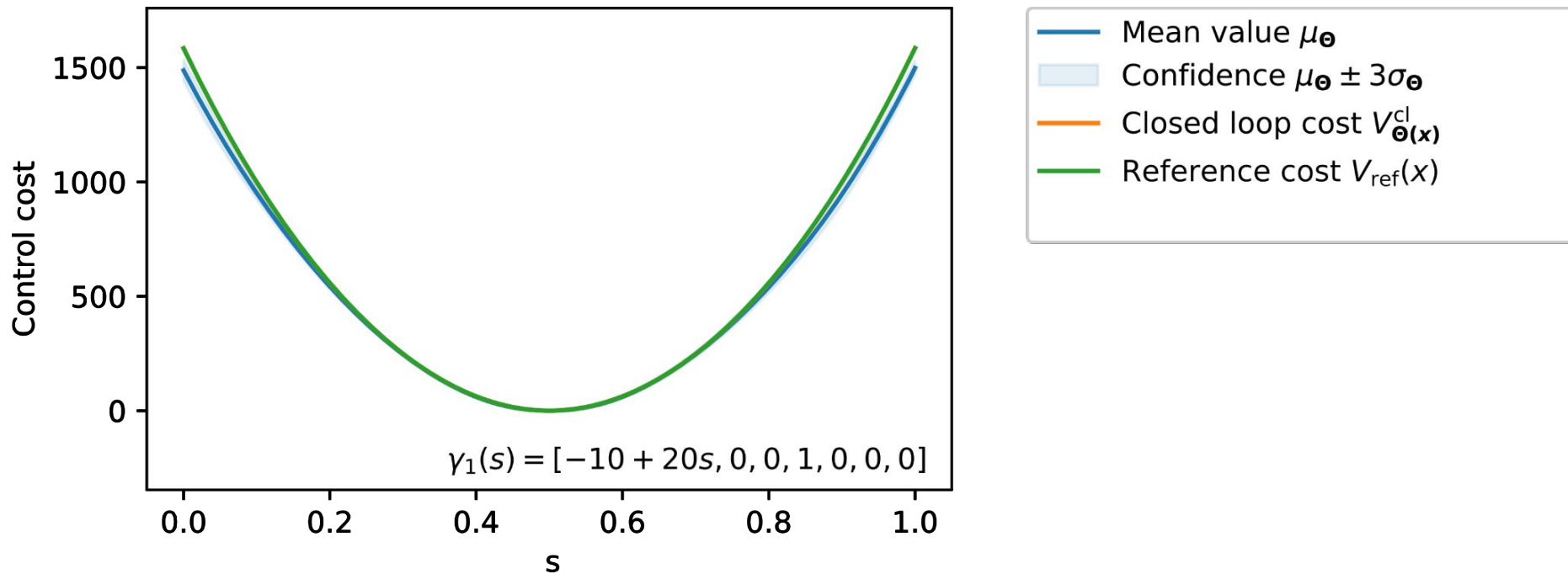
- Function approximation with uncertainty estimate
 - Simple NN ensemble
- Uncertainty sampling
 - Generate new data point where uncertainty estimate is high
- Uncertainty threshold
 - Every point with uncertainty below threshold is considered “known”

6D “Flat quad” system

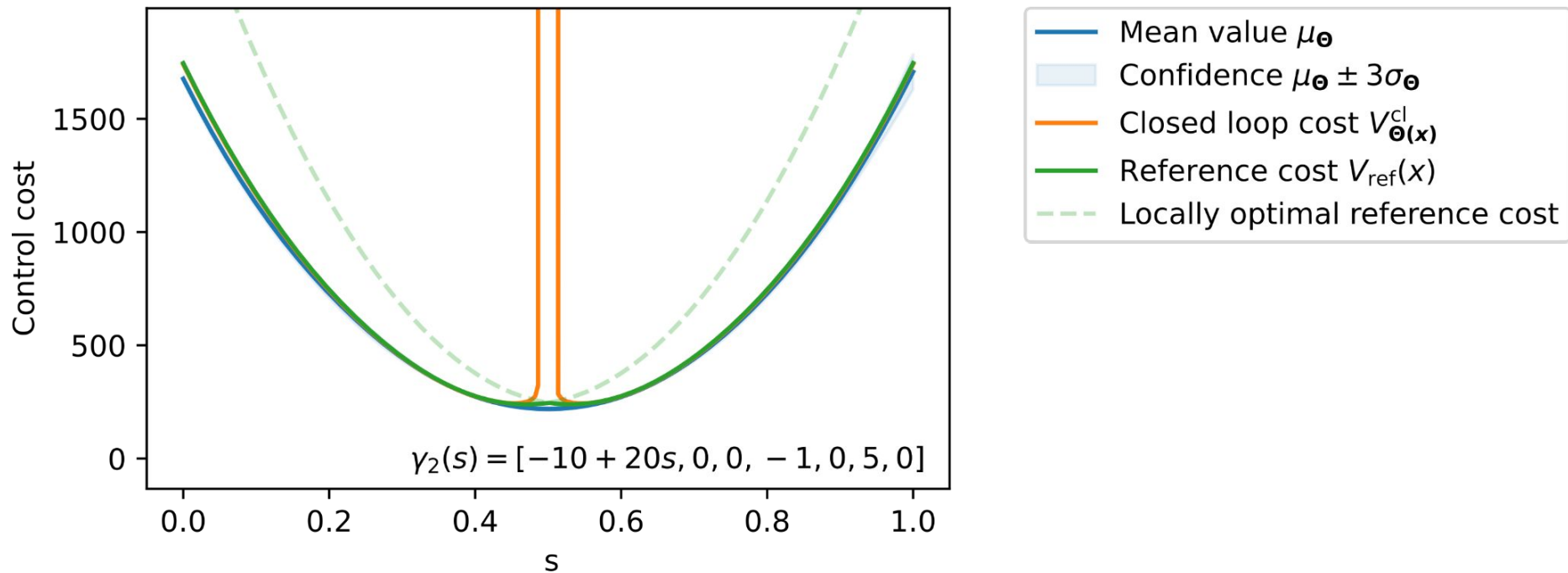


- $\theta \rightarrow (\sin \theta, \cos \theta)$ to avoid wrapping
- 6D submanifold of $\mathbb{R}^7$

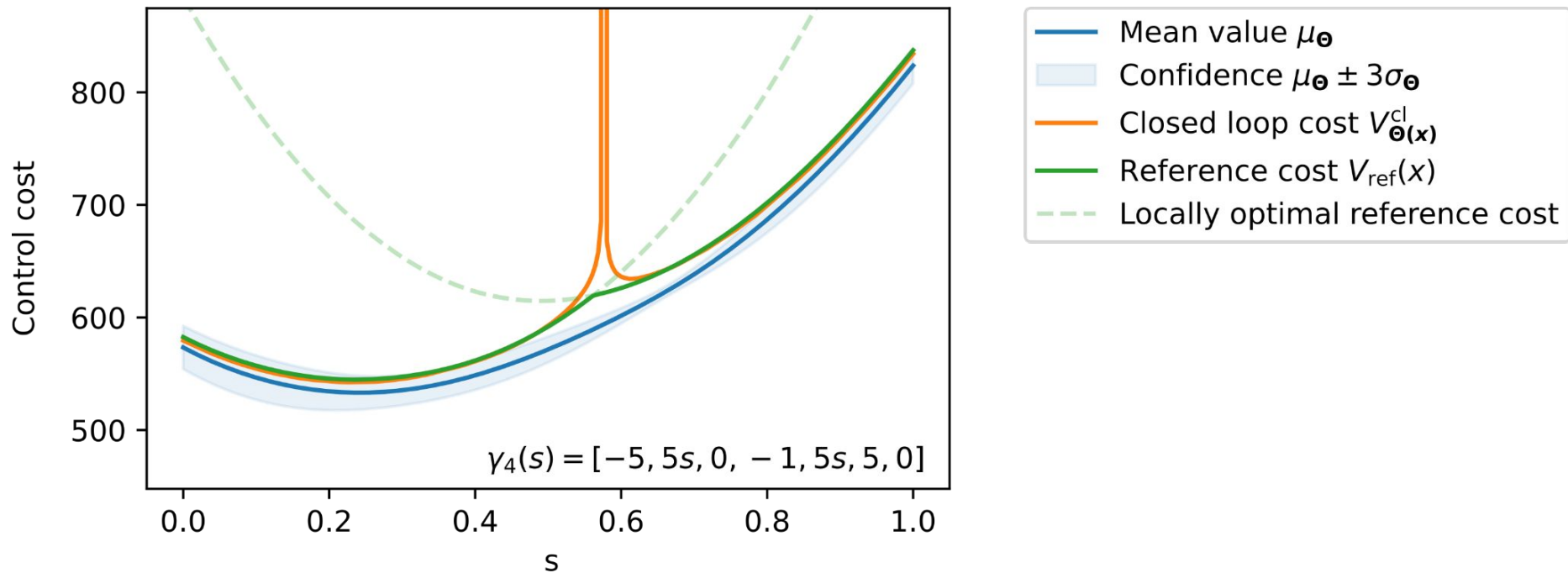
Results (6D system)



Results (6D system)



Results (6D system)



Conclusions

- Proposed method does accurately solve continuous-time problems
- Accuracy deteriorates where V is nonsmooth and control discontinuous
- Further Research:
 - Function approximation capable of nonsmoothness
 - More general problem settings (State constraints, robust control, ...)
 - Scalability to larger systems, computational efficiency
 - Concrete error analysis

Questions?



Bonus material



Algorithm 1 Active Learning formulation of Level-Set DP

Require: Initial approximate value function $V^\Theta = (\mu_0^\Theta, \sigma_0^\Theta)$

Require: initial dataset $\mathcal{D}$

- 1: **for** $k \in \mathbb{N}$ **do**
 - 2: $v_k \leftarrow \text{EstimateValueLevel}(V_k^\Theta)$
 - 3: **if** $v_k > V_{\max}$ **then** Return V_k^Θ
 - 4: $\hat{v}_{k+1} \leftarrow \text{SetValueTarget}(V_k^\Theta, v_k)$
 - 5: $\hat{\mathcal{X}} \leftarrow \text{ProposePoints}(\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1})$
 - 6: $\mathcal{D} \leftarrow \mathcal{D} \cup \text{GenerateTrajectories}(\hat{\mathcal{X}}, V_k^\Theta, v_k)$
 - 7: $\mathcal{D}, V_{k+1}^\Theta \leftarrow \text{TrainAndPrune}(\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1})$
 - 8: **end for**
-

Algorithm 3 Batch Proposal Selection

function PROPOSEPOINTS($\mathcal{D}, V_k^\Theta, v_k, \hat{v}_{k+1}$)

$\hat{\mathcal{X}} \leftarrow \emptyset$

$(\mu^\Theta, \sigma^\Theta) \leftarrow \mathcal{V}_k^\Theta$

$\sigma_p \leftarrow \sigma^\Theta$

▷ Update proxy uncertainty based on existing data

for $(x, \lambda, v) \in \mathcal{D}$ **do**

if $v \in [v_k, \hat{v}_{k+1}]$ **then**

$\sigma_p \leftarrow (x \mapsto \sigma_p(x)(1 - k(\hat{x}, x)))$

end if

end for

▷ Propose new points sequentially

for $i = 0, \dots, N_{\text{batch}}$ **do**

$f_p \leftarrow (x \mapsto \sigma_p(x)/\sigma_{\max}(x))$ ▷ Define proxy acquisition function

$\hat{x} \leftarrow \operatorname{argmax}_{x: \mu^\Theta(x) \in [v_k, \hat{v}_{k+1}]} f_p(x)$ ▷ Find proposal

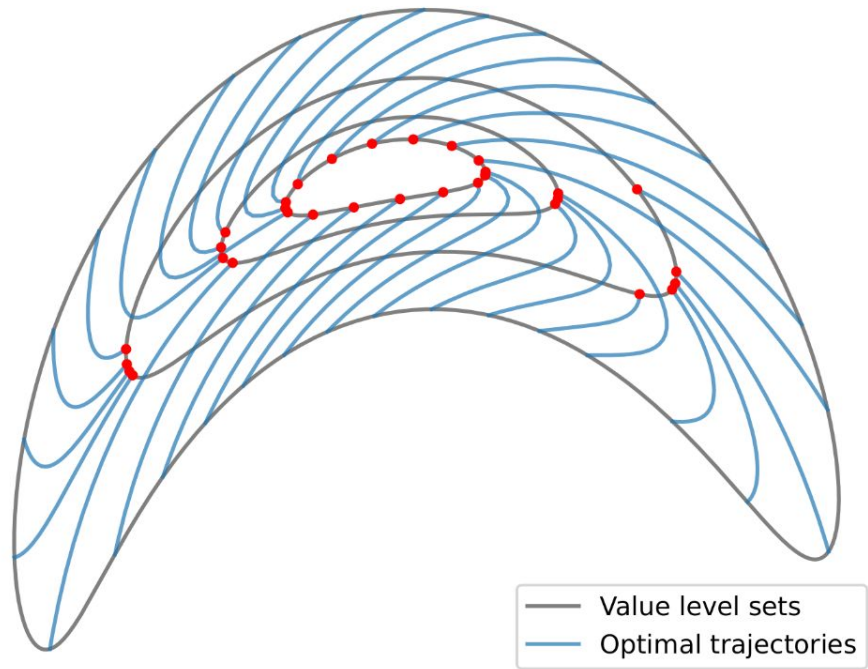
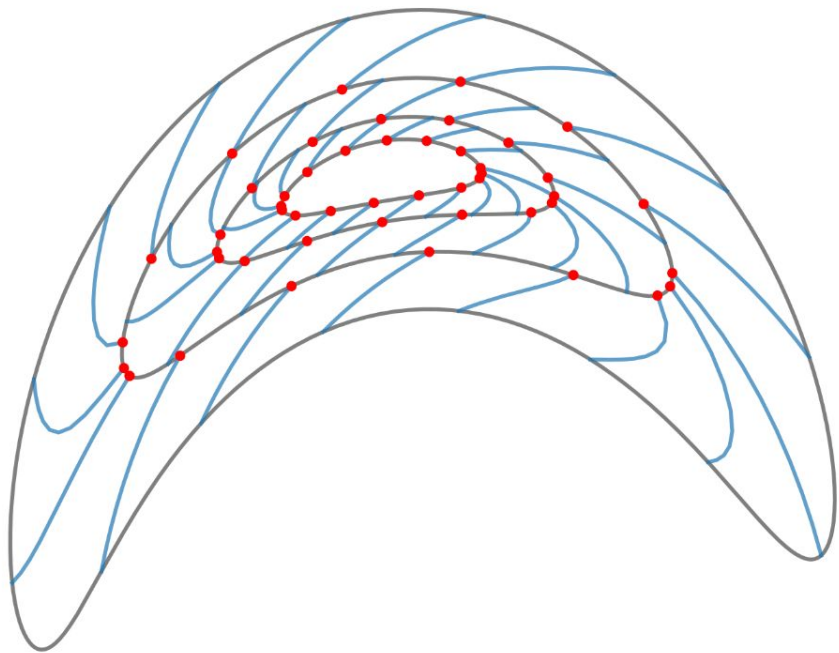
$\hat{\mathcal{X}} \leftarrow \hat{\mathcal{X}} \cup \{\hat{x}\}$

$\sigma_p \leftarrow (x \mapsto \sigma_p(x)(1 - k(\hat{x}, x)))$ ▷ Update proxy uncertainty

end for

return $\hat{\mathcal{X}}$

end function



— Value level sets
— Optimal trajectories

Results (2D system)

